Setor de Educação Profissional e Tecnológica
Tecnologia em Análise e Desenvolvimento de Sistemas

DS143 - Estruturas de Dados II

# 13 - Árvores geradoras mínimas

Prof. Alex Kutzke

Novembro de 2026

# Objetivos da aula

Ao final desta aula você deve ser capaz de:

1. Definir árvore geradora e árvore geradora mínima de um grafo com pesos;
2. Enunciar a propriedade do corte e usá-la para mostrar que uma aresta pertence à árvore mínima, indicando um corte em que ela é a aresta de travessia mais leve;
3. Executar à mão os algoritmos de Kruskal e de Prim, registrando a ordem em que as arestas entram e as que são descartadas;
4. Explicar o papel do Union-Find no Kruskal e o da fila de prioridade no Prim, e o custo de cada algoritmo;
5. Discutir o que a árvore geradora mínima modela bem num projeto de infraestrutura, e o que ela deixa de fora.

Este conteúdo cai na Prova 2.

# Roteiro

Árvores geradoras

Kruskal

Prim

Infraestrutura

Fechamento

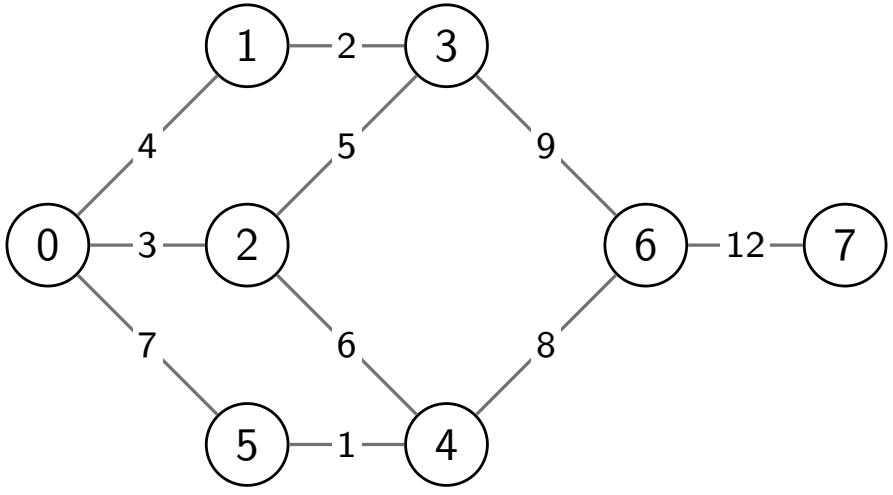
# Demonstração: fibra óptica para escolas rurais

```
$ go run escolas.go
Tudo ligado à sede                    Árvore geradora mínima
|            K              .G   |   |            .K....        ...G   |
|   A.         .      F   ..     |   |   A..    ..       ...F...        |
|     ...B.    .      . ...      |   |       ..B.                       |
|        ....  .  ....           |   |           ..                     |
|           .... ...             |   |             ..                   |
|             ..S...             |   |               .S...              |
|        ....... .......H        |   |                 .    ....H       |
|     C....    .  .    ...       |   |     C.         .           ..     |
|   ....       .    .      ....   |   |   ..  ...  .                ..   |
| D.         E     .           .I |   | D.        .E...                .I |
|                   J            |   |                ...J              |
11 ligações, 129.8 km                 11 ligações, 81.6 km
```

- município fictício: a sede S já tem conexão, as escolas A a K não;
- as duas redes ligam todas as escolas; a da direita usa 63% do cabo.

# Grafo com pesos

Cada aresta tem um **peso**: distância, custo, tempo.




- o grafo $G$ da aula de representação e percursos, agora com pesos;
- os pesos são números quaisquer, inclusive negativos ou repetidos.

# Representação em Go

```
type Aresta struct {
  V, W int
  Peso float64
}

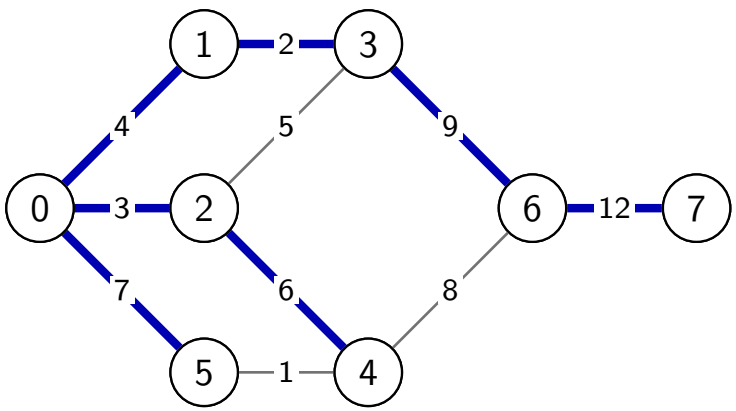
// Outro devolve o extremo da aresta que não é v.
func (a Aresta) Outro(v int) int {
  if v == a.V {
    return a.W
  }
  return a.V
}

type GrafoP struct {
  adj [][]Aresta // a aresta v-w está na lista de v e na de w
}
```

As listas de adjacência guardam a aresta inteira, e não só o vizinho.

# Árvore geradora

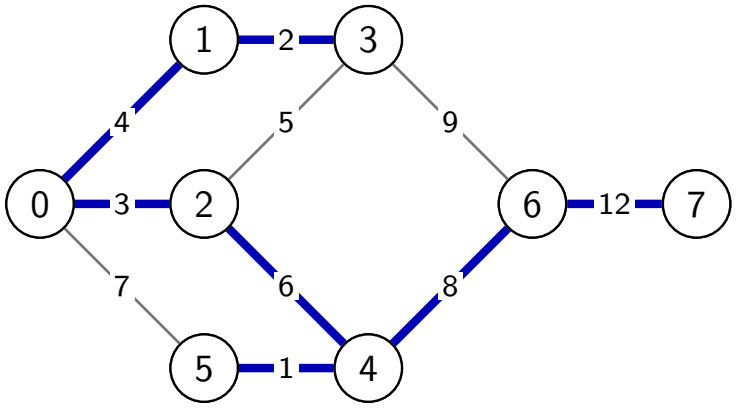
Uma **árvore geradora** de um grafo conexo é um subconjunto das arestas que liga todos os vértices e não tem ciclo.




- tem sempre $V - 1$ arestas;
- a árvore da busca em largura é uma árvore geradora;
- à esquerda, a de $G$ a partir de 0, com peso 43:
  $4 + 3 + 7 + 2 + 6 + 9 + 12$.

# Árvore geradora mínima

A **árvore geradora mínima** (AGM, *minimum spanning tree*) é a árvore geradora de menor peso total.

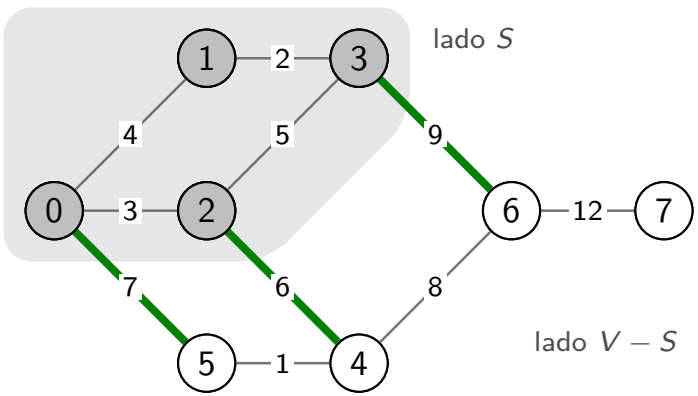



- em $G$, peso 36;
- $G$ tem 50 árvores geradoras; o grafo completo com 12 vértices da demonstração tem $12^{10}$, cerca de 62 bilhões;
- testar todas está fora de questão.

Com pesos distintos, a árvore mínima é única. Com pesos repetidos, pode haver mais de uma, todas com o mesmo peso.

# Propriedade do corte

Um **corte** divide os vértices em dois conjuntos não vazios. Uma **aresta de travessia** liga um vértice de cada lado.




Arestas de travessia: 0-5 (7), 2-4 (6) e 3-6 (9).

**Propriedade do corte**: em qualquer corte, a aresta de travessia de menor peso está na árvore geradora mínima (pesos distintos).

# Por que a propriedade do corte vale

Seja $e$ a aresta de travessia mais leve, e suponha uma árvore mínima $T$ sem $e$.

1. acrescentar $e$ a $T$ fecha um ciclo;
2. o ciclo atravessa o corte por $e$ e precisa voltar: alguma outra aresta $f$ do ciclo também é de travessia;
3. trocar $f$ por $e$ dá outra árvore geradora;
4. $e$ pesa menos que $f$, então a árvore nova pesa menos que $T$.

Contradição: $T$ não era mínima.

No corte do slide anterior: sem 2-4, a árvore teria de usar 0-5 ou 3-6, e trocar essa aresta por 2-4 diminuiria o peso.

# Algoritmo guloso

1. comece sem nenhuma aresta;
2. escolha um corte que nenhuma aresta já escolhida atravessa;
3. acrescente a aresta de travessia mais leve desse corte;
4. repita até ter $V - 1$ arestas.

Cada aresta acrescentada está na árvore mínima, pela propriedade do corte. Kruskal e Prim são duas formas de escolher o corte a cada passo.

# Algoritmo de Kruskal

1. ordene as arestas por peso, da mais leve para a mais pesada;
2. examine cada aresta $v$-$w$ nessa ordem:
   - se $v$ e $w$ já estão ligados pelas arestas escolhidas, a aresta fecharia um ciclo: descarte;
   - se não, a aresta entra;
3. pare com $V - 1$ arestas.

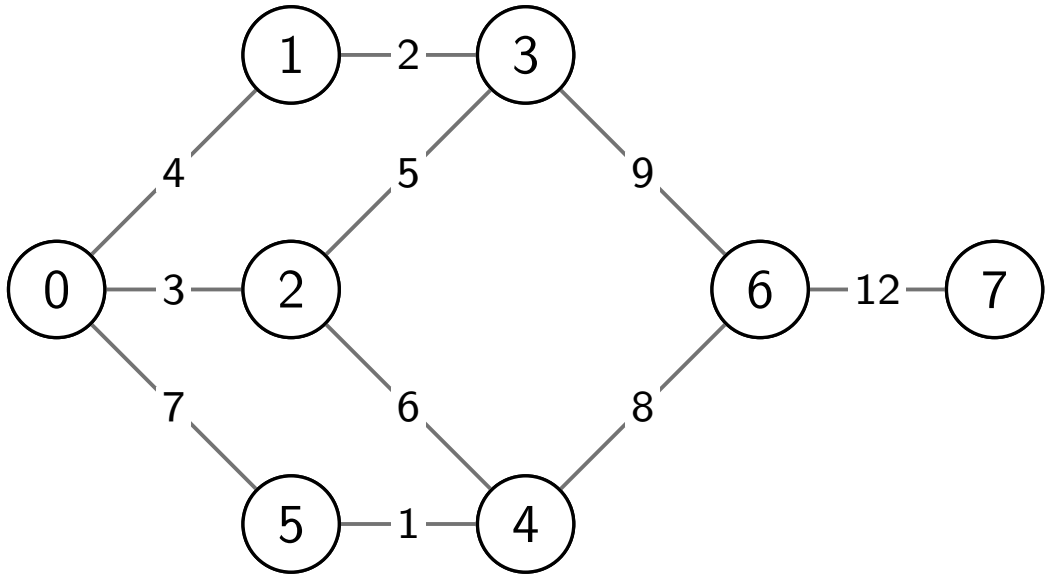
"$v$ e $w$ já estão ligados?" é a pergunta do Union-Find, a primeira estrutura do semestre: `Connected` e `Union`, na versão com união por tamanho.

# Kruskal em Go

```go
func Kruskal(n int, arestas []Aresta) []Aresta {
  ordem := make([]Aresta, len(arestas))
  copy(ordem, arestas)
  sort.Slice(ordem, func(i, j int) bool {
    return ordem[i].Peso < ordem[j].Peso
  })

  uf := NovoUF(n)
  var arvore []Aresta
  for _, a := range ordem {
    if len(arvore) == n-1 {
      break // a árvore já tem V-1 arestas
    }
    if uf.Connected(a.V, a.W) {
      continue // fecharia um ciclo
    }
    uf.Union(a.V, a.W)
    arvore = append(arvore, a)
  }
  return arvore
}
```

# Kruskal passo a passo



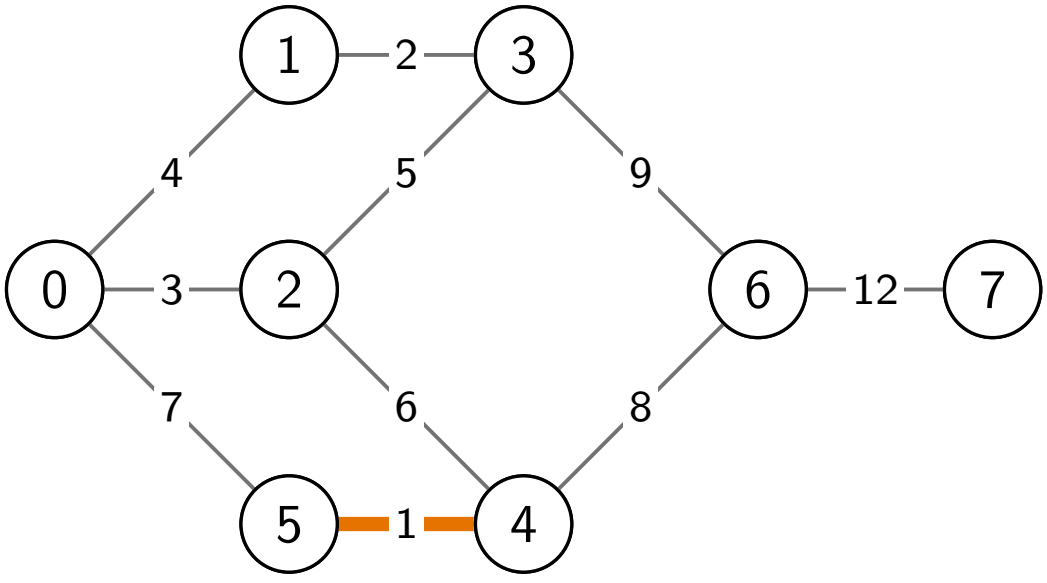

| aresta | situação |
|---|---|
| 4-5 | 1 |
| 1-3 | 2 |
| 0-2 | 3 |
| 0-1 | 4 |
| 2-3 | 5 |
| 2-4 | 6 |
| 0-5 | 7 |
| 4-6 | 8 |
| 3-6 | 9 |
| 6-7 | 12 |

componentes: {0} {1} {2} {3} {4} {5} {6} {7}

início: arestas em ordem de peso; cada vértice é uma componente

na árvore  entra agora  descartada: fecharia um ciclo

# Kruskal passo a passo



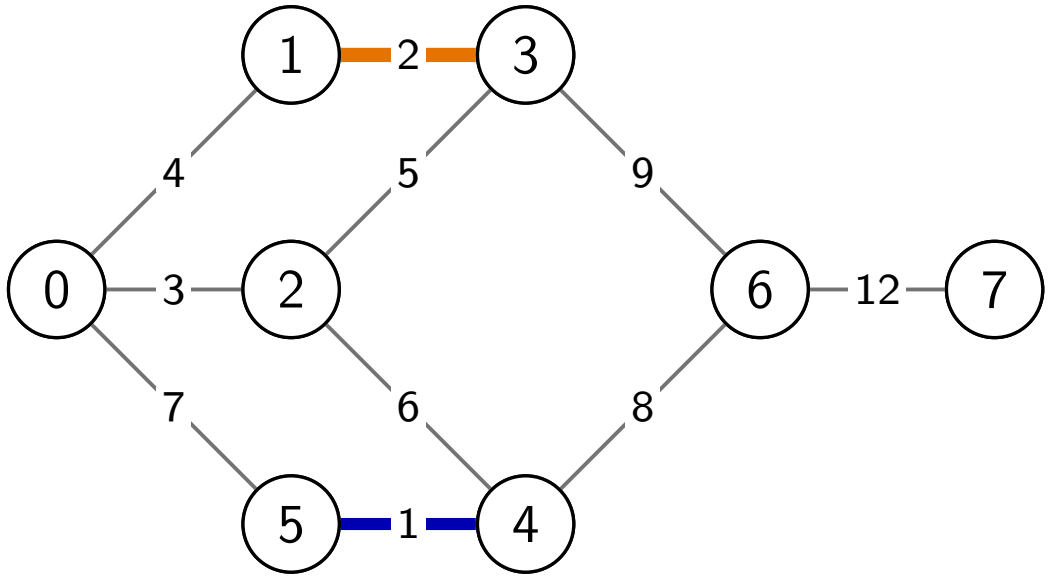

| aresta | | situação |
|---|---|---|
| 4-5 | 1 | entra |
| 1-3 | 2 | |
| 0-2 | 3 | |
| 0-1 | 4 | |
| 2-3 | 5 | |
| 2-4 | 6 | |
| 0-5 | 7 | |
| 4-6 | 8 | |
| 3-6 | 9 | |
| 6-7 | 12 | |

componentes: {0} {1} {2} {3} {4 5} {6} {7}

4-5, peso 1: 4 e 5 em componentes diferentes, entra

na árvore  entra agora  descartada: fecharia um ciclo

# Kruskal passo a passo



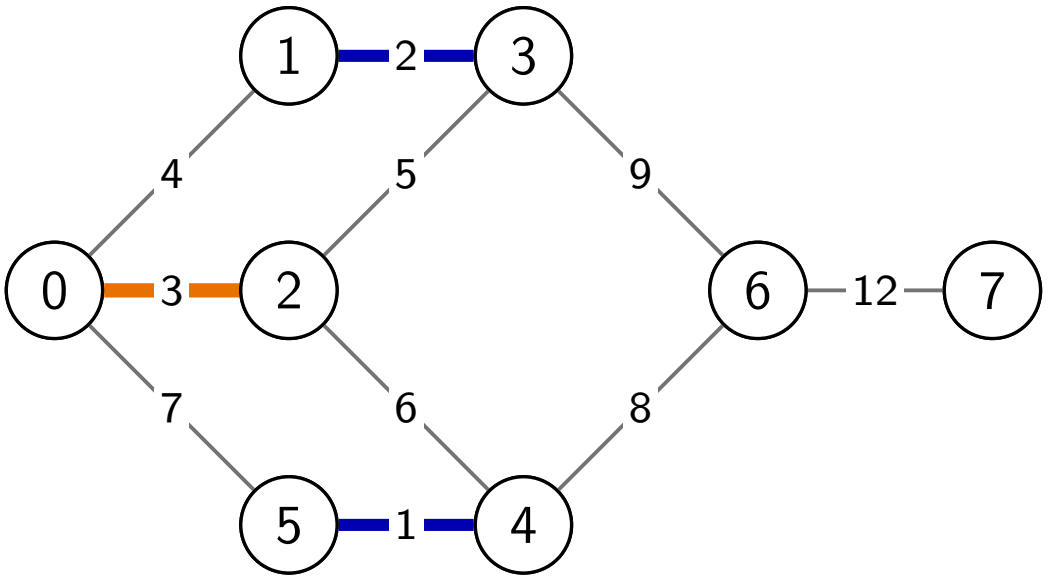

| aresta | | situação |
|---|---|---|
| 4–5 | 1 | entra |
| 1–3 | 2 | entra |
| 0–2 | 3 | |
| 0–1 | 4 | |
| 2–3 | 5 | |
| 2–4 | 6 | |
| 0–5 | 7 | |
| 4–6 | 8 | |
| 3–6 | 9 | |
| 6–7 | 12 | |

componentes: {0} {1 3} {2} {4 5} {6} {7}

1-3, peso 2: 1 e 3 em componentes diferentes, entra

na árvore  entra agora  descartada: fecharia um ciclo

# Kruskal passo a passo



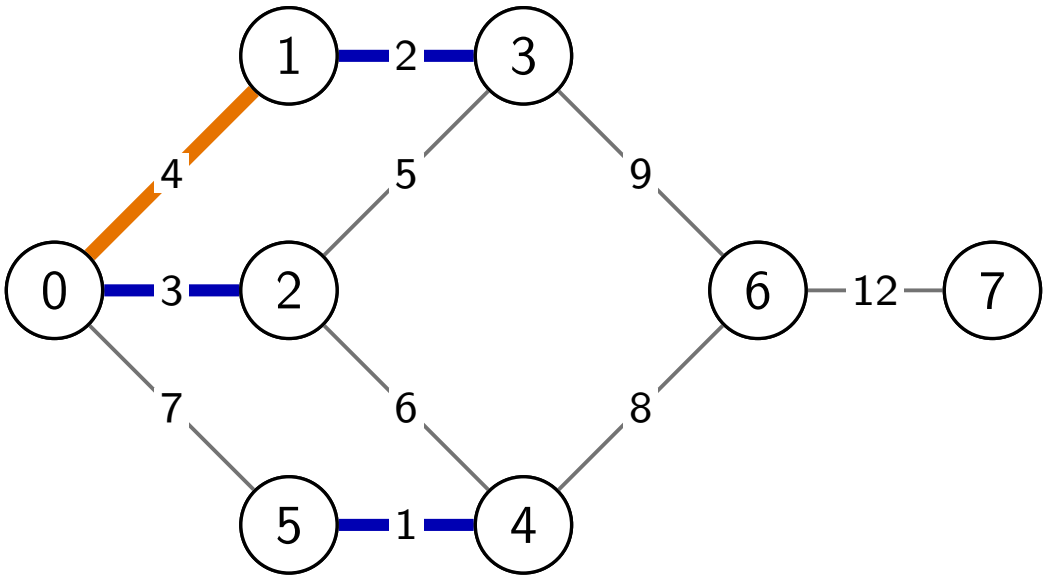

| aresta | | situação |
|---|---|---|
| 4–5 | 1 | entra |
| 1–3 | 2 | entra |
| 0–2 | 3 | entra |
| 0–1 | 4 | |
| 2–3 | 5 | |
| 2–4 | 6 | |
| 0–5 | 7 | |
| 4–6 | 8 | |
| 3–6 | 9 | |
| 6–7 | 12 | |

componentes: {0 2} {1 3} {4 5} {6} {7}

0-2, peso 3: 0 e 2 em componentes diferentes, entra

na árvore  entra agora  descartada: fecharia um ciclo

# Kruskal passo a passo



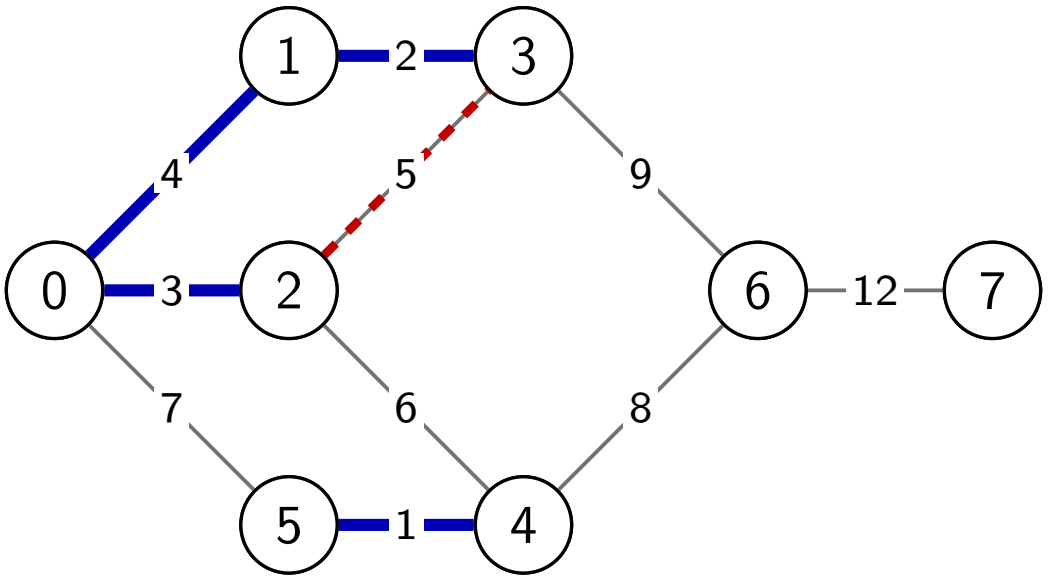

| aresta | | situação |
|---|---|---|
| 4–5 | 1 | entra |
| 1–3 | 2 | entra |
| 0–2 | 3 | entra |
| 0–1 | 4 | entra |
| 2–3 | 5 | |
| 2–4 | 6 | |
| 0–5 | 7 | |
| 4–6 | 8 | |
| 3–6 | 9 | |
| 6–7 | 12 | |

componentes: {0 1 2 3} {4 5} {6} {7}

0-1, peso 4: 0 e 1 em componentes diferentes, entra

na árvore  entra agora  descartada: fecharia um ciclo

# Kruskal passo a passo



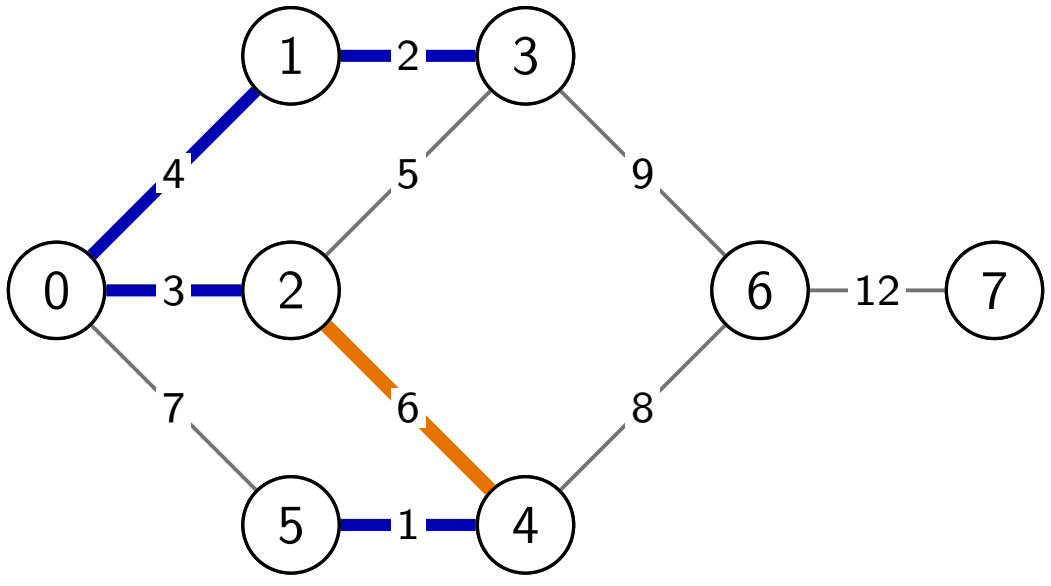

| aresta | | situação |
|---|---|---|
| 4–5 | 1 | entra |
| 1–3 | 2 | entra |
| 0–2 | 3 | entra |
| 0–1 | 4 | entra |
| 2–3 | 5 | descarta |
| 2–4 | 6 | |
| 0–5 | 7 | |
| 4–6 | 8 | |
| 3–6 | 9 | |
| 6–7 | 12 | |

componentes: {0 1 2 3} {4 5} {6} {7}

2-3, peso 5: 2 e 3 na mesma componente, descarta

na árvore  entra agora  descartada: fecharia um ciclo

# Kruskal passo a passo



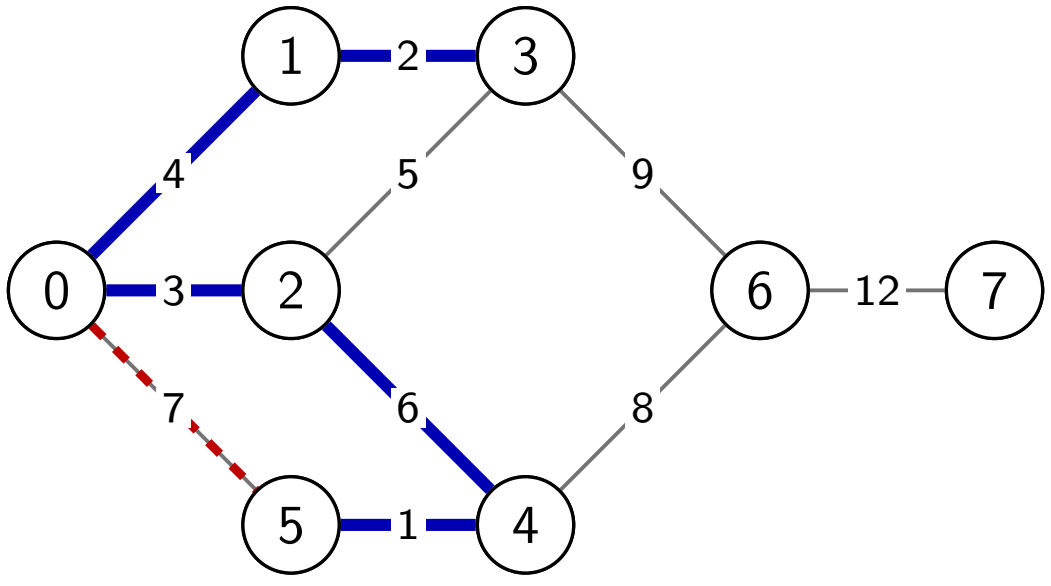

| aresta | | situação |
|---|---|---|
| 4–5 | 1 | entra |
| 1–3 | 2 | entra |
| 0–2 | 3 | entra |
| 0–1 | 4 | entra |
| 2–3 | 5 | descarta |
| 2–4 | 6 | entra |
| 0–5 | 7 | |
| 4–6 | 8 | |
| 3–6 | 9 | |
| 6–7 | 12 | |

componentes: {0 1 2 3 4 5} {6} {7}

2-4, peso 6: 2 e 4 em componentes diferentes, entra

na árvore  entra agora  descartada: fecharia um ciclo

# Kruskal passo a passo



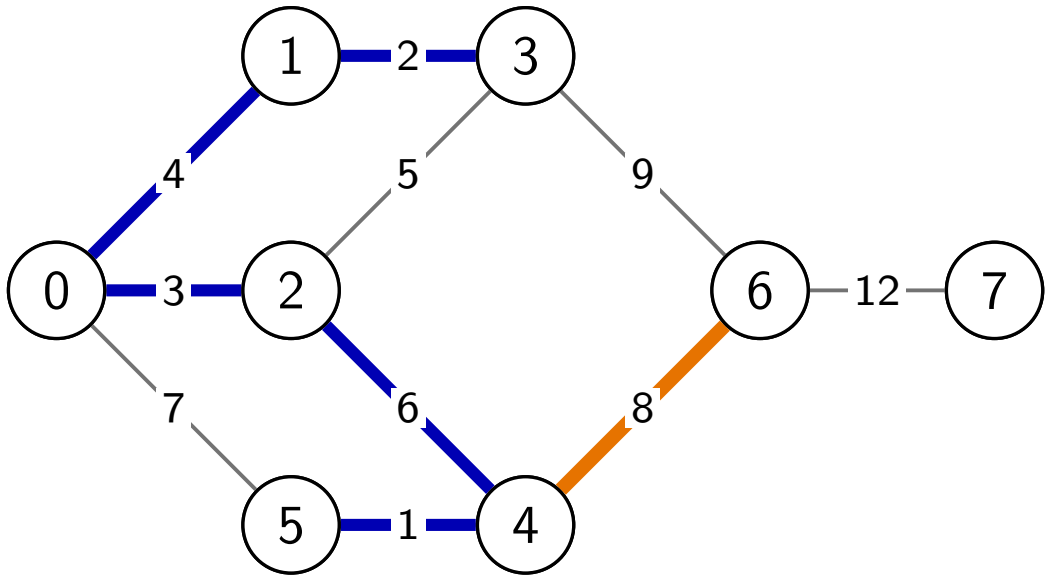

| aresta | | situação |
|---|---|---|
| 4–5 | 1 | entra |
| 1–3 | 2 | entra |
| 0–2 | 3 | entra |
| 0–1 | 4 | entra |
| 2–3 | 5 | descarta |
| 2–4 | 6 | entra |
| 0–5 | 7 | descarta |
| 4–6 | 8 | |
| 3–6 | 9 | |
| 6–7 | 12 | |

componentes: {0 1 2 3 4 5} {6} {7}

0-5, peso 7: 0 e 5 na mesma componente, descarta

na árvore  entra agora  descartada: fecharia um ciclo

# Kruskal passo a passo



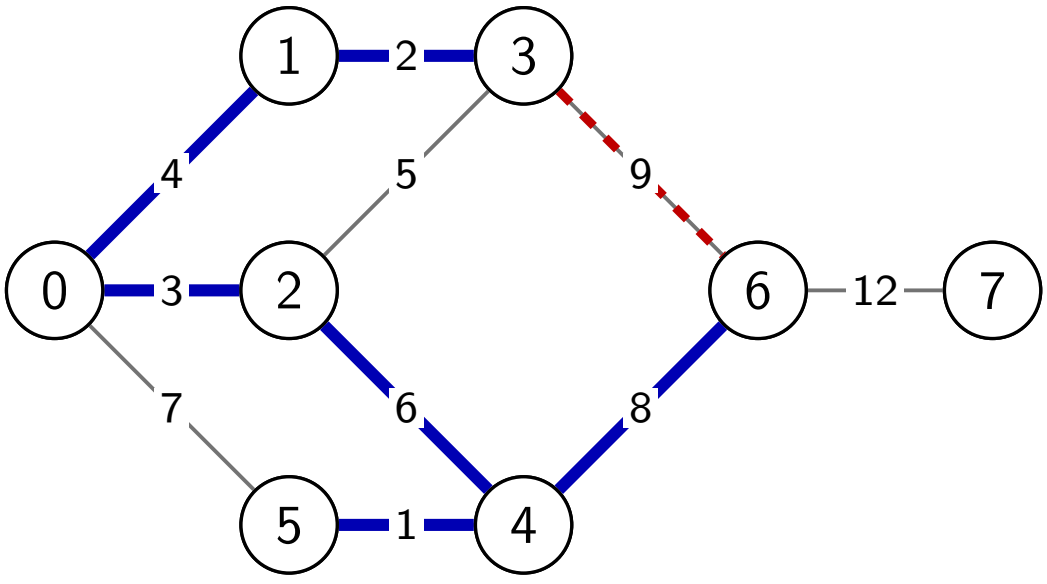

| aresta | | situação |
|---|---|---|
| 4–5 | 1 | entra |
| 1–3 | 2 | entra |
| 0–2 | 3 | entra |
| 0–1 | 4 | entra |
| 2–3 | 5 | descarta |
| 2–4 | 6 | entra |
| 0–5 | 7 | descarta |
| 4–6 | 8 | entra |
| 3–6 | 9 | |
| 6–7 | 12 | |

componentes: {0 1 2 3 4 5 6} {7}

4-6, peso 8: 4 e 6 em componentes diferentes, entra

na árvore  entra agora  descartada: fecharia um ciclo

# Kruskal passo a passo



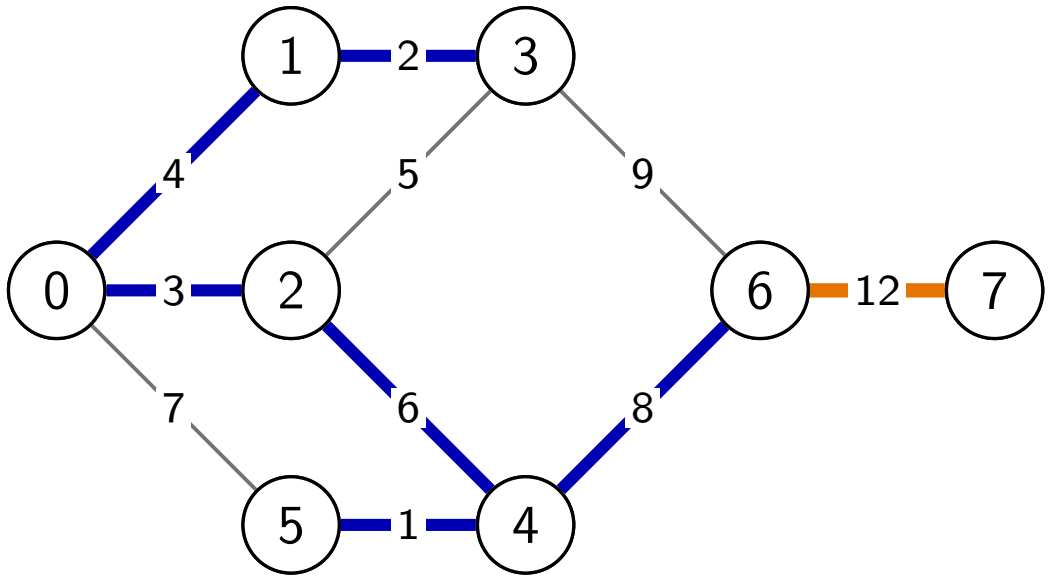

| aresta | | situação |
|---|---|---|
| 4–5 | 1 | entra |
| 1–3 | 2 | entra |
| 0–2 | 3 | entra |
| 0–1 | 4 | entra |
| 2–3 | 5 | descarta |
| 2–4 | 6 | entra |
| 0–5 | 7 | descarta |
| 4–6 | 8 | entra |
| 3–6 | 9 | descarta |
| 6–7 | 12 | |

componentes: {0 1 2 3 4 5 6} {7}

3-6, peso 9: 3 e 6 na mesma componente, descarta

na árvore  entra agora  descartada: fecharia um ciclo

# Kruskal passo a passo




| aresta | | situação |
|---|---|---|
| 4–5 | 1 | entra |
| 1–3 | 2 | entra |
| 0–2 | 3 | entra |
| 0–1 | 4 | entra |
| 2–3 | 5 | descarta |
| 2–4 | 6 | entra |
| 0–5 | 7 | descarta |
| 4–6 | 8 | entra |
| 3–6 | 9 | descarta |
| 6–7 | 12 | entra |

componentes: {0 1 2 3 4 5 6 7}

6-7, peso 12: entra; a árvore tem $V - 1 = 7$ arestas, fim

na árvore  entra agora  descartada: fecharia um ciclo

# Kruskal em $G$

```
$ go run kruskal.go
Kruskal: arestas da mais leve para a mais pesada
  4-5     1   entra
  1-3     2   entra
  0-2     3   entra
  0-1     4   entra
  2-3     5   descarta: 2 e 3 já estão ligados
  2-4     6   entra
  0-5     7   descarta: 0 e 5 já estão ligados
  4-6     8   entra
  3-6     9   descarta: 3 e 6 já estão ligados
  6-7    12   entra

Árvore geradora mínima: 4-5 1-3 0-2 0-1 2-4 4-6 6-7
7 arestas, peso total 36
```

Custo: ordenar as arestas, $O(E \log E)$, mais até $E$ consultas ao Union-Find, $O(\log V)$ cada. Total $O(E \log E)$.

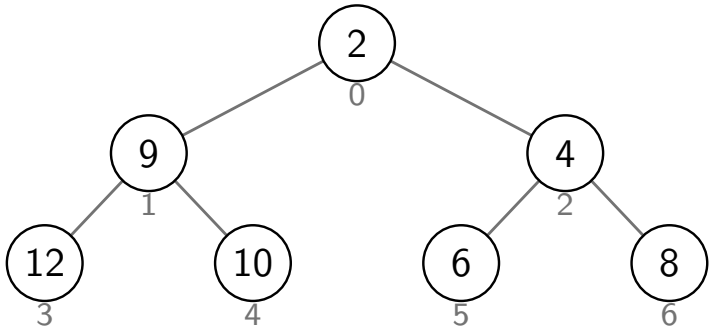
# Algoritmo de Prim

1. comece a árvore por um vértice $s$;
2. acrescente a aresta mais leve que liga a árvore a um vértice fora dela, e esse vértice;
3. repita até a árvore ter todos os vértices.

O corte de cada passo é $\{$vértices da árvore$\}$ e $\{$os demais$\}$.

Para achar a aresta mais leve sem percorrer todas a cada passo, as candidatas ficam numa **fila de prioridade**.

# Heap de mínimo




| 2 | 9 | 4 | 12 | 10 | 6 | 8 |
|---|---|---|---|---|---|---|
| 0 | 1 | 2 | 3 | 4 | 5 | 6 |

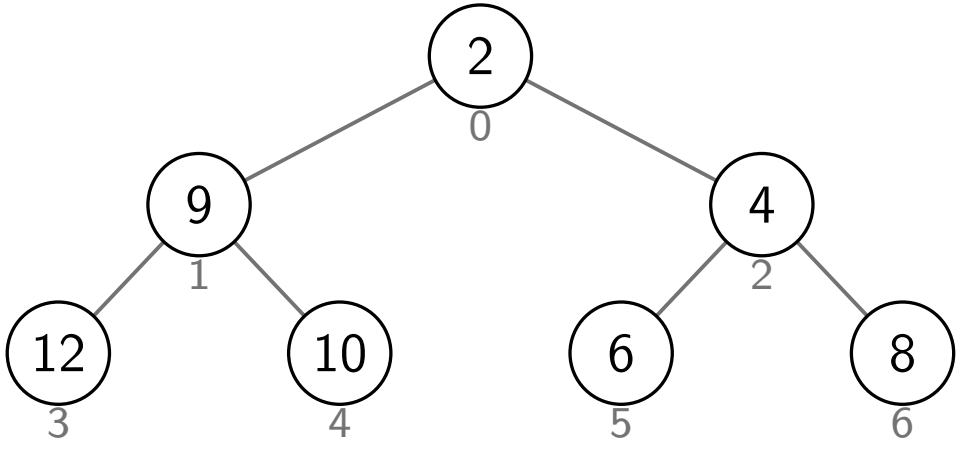
Heap de mínimo com 7 elementos: cada um é menor ou igual aos filhos.

Fila de prioridade: insere em qualquer ordem e retira sempre o menor.

- árvore binária completa, com os níveis preenchidos da esquerda para a direita;
- cada elemento é menor ou igual aos filhos, então o menor está na raiz;
- guardada num vetor: os filhos de $i$ estão em $2i + 1$ e $2i + 2$, e o pai em $(i - 1)/2$.

A altura é $\lfloor \log_2 n \rfloor$: inserir e retirar o menor custam $O(\log n)$.

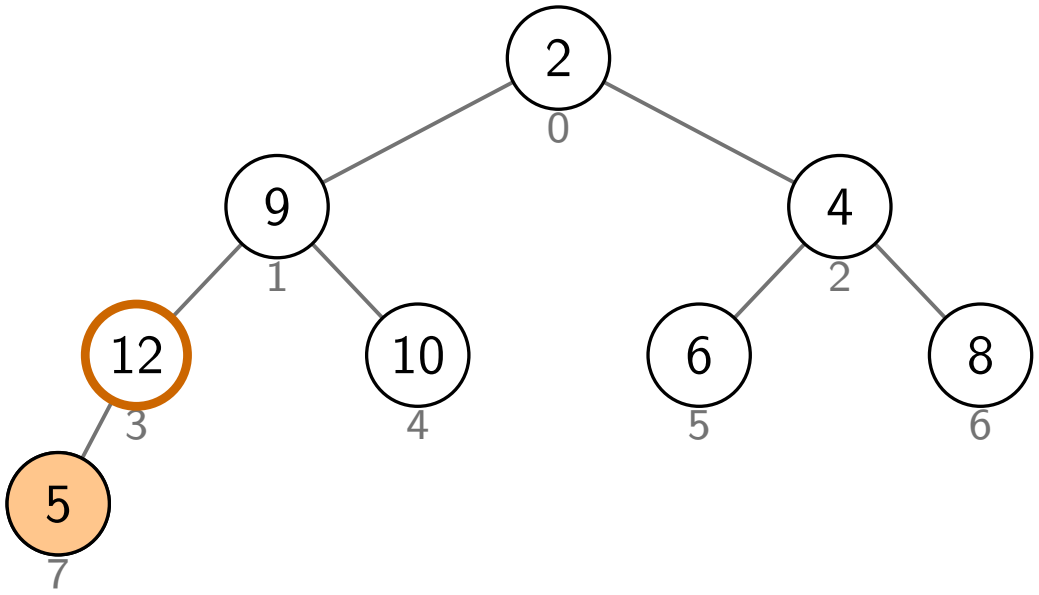
# Inserção: o novo elemento sobe




| 2 | 9 | 4 | 12 | 10 | 6 | 8 |
|---|---|---|---|---|---|---|
| 0 | 1 | 2 | 3 | 4 | 5 | 6 |

Heap de mínimo com 7 elementos:
cada um é menor ou igual aos filhos.

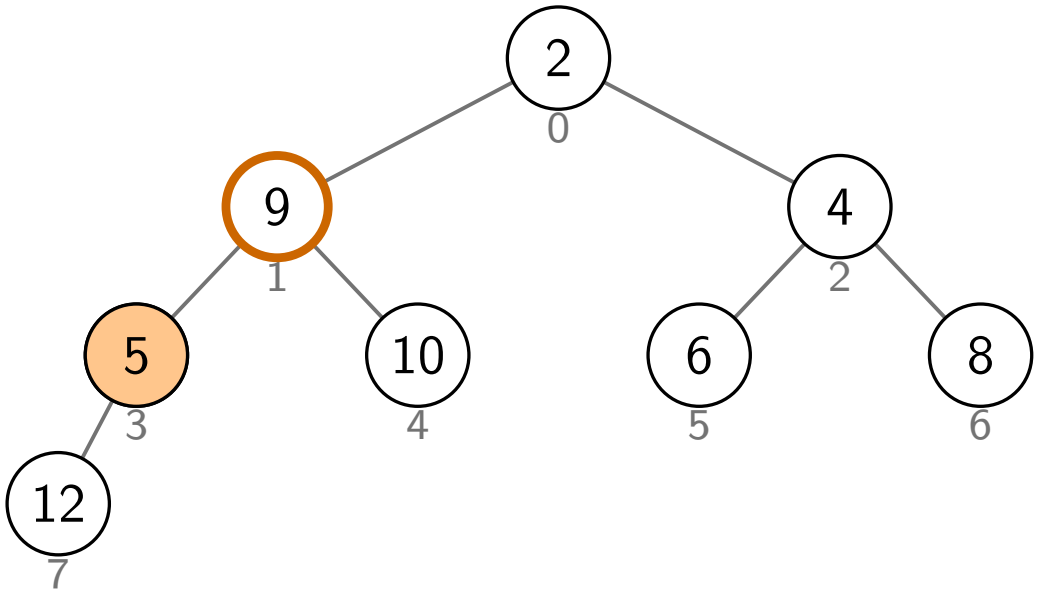
# Inserção: o novo elemento sobe




| 2 | 9 | 4 | 12 | 10 | 6 | 8 | 5 |
|---|---|---|---|---|---|---|---|
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |

Insere o 5: ele entra no fim do vetor, no índice 7. O pai, no índice 3, é 12. Como $5 < 12$, os dois trocam.

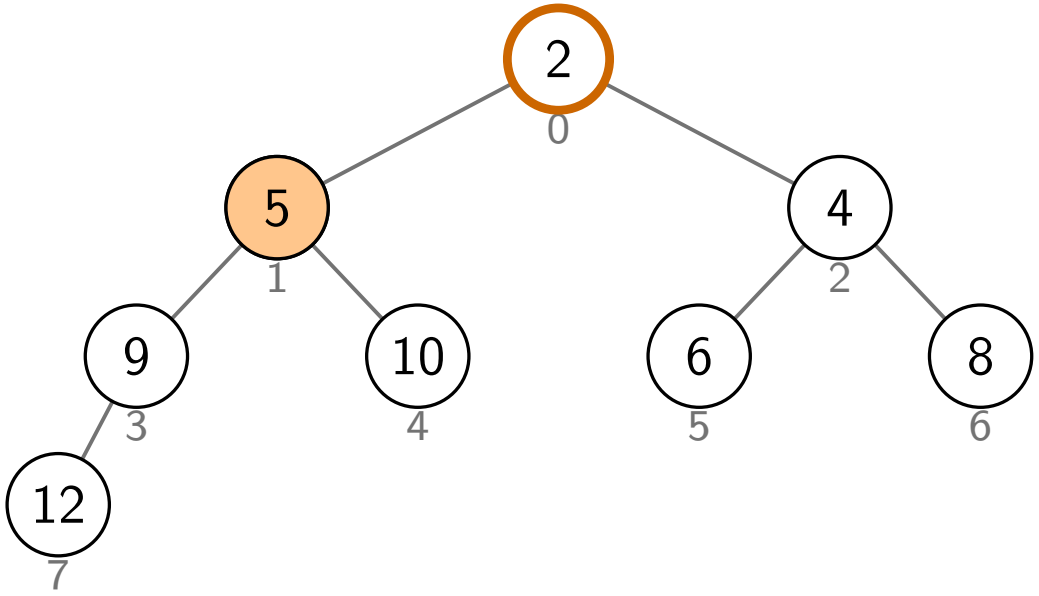
# Inserção: o novo elemento sobe




| 2 | 9 | 4 | 5 | 10 | 6 | 8 | 12 |
|---|---|---|---|---|---|---|---|
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |

O pai do índice 3 é o índice 1, com 9. Como $5 < 9$, os dois trocam.

# Inserção: o novo elemento sobe




O pai do índice 1 é a raiz, com 2. Como $2 \leq 5$, o 5 para. Foram duas trocas.

# Inserção em Go

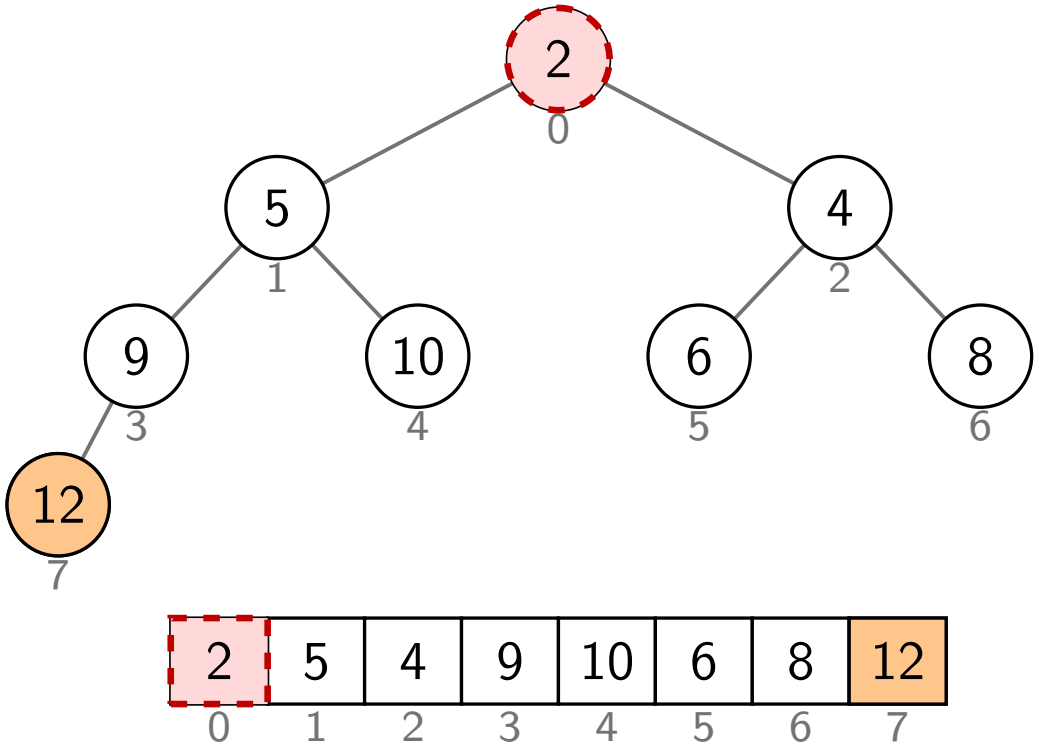
```
type HeapMin struct {
  v []int
}

func (h *HeapMin) Insere(x int) {
  h.v = append(h.v, x)
  h.sobe(len(h.v) - 1)
}
```

```
func (h *HeapMin) sobe(i int) {
  for i > 0 {
    pai := (i - 1) / 2
    if h.v[pai] <= h.v[i] {
      return
    }
    h.v[pai], h.v[i] = h.v[i], h.v[pai]
    i = pai
  }
}
```

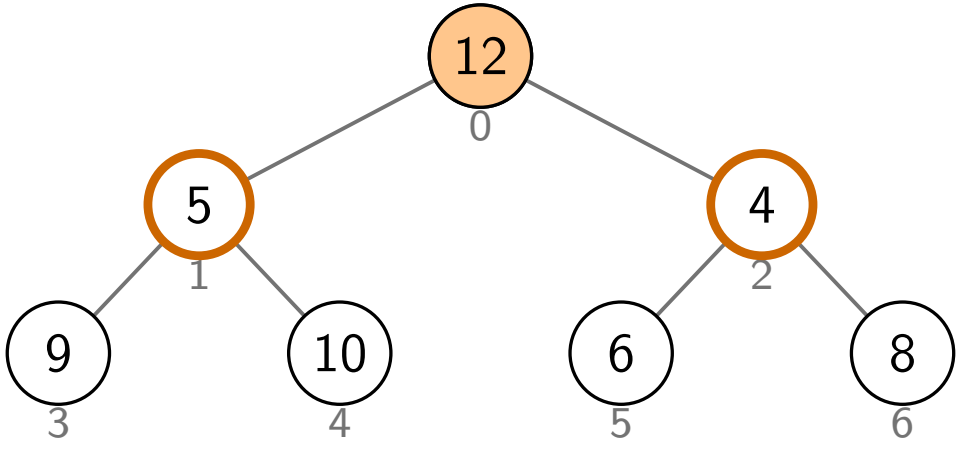
O novo elemento entra no fim do vetor e troca de lugar com o pai enquanto for menor que ele. São no máximo $\lfloor \log_2 n \rfloor$ trocas, uma por nível.

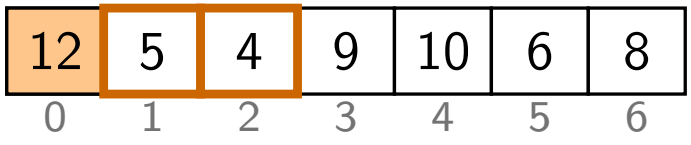
# Retirada do mínimo: o último elemento desce




Retira o menor, o 2, que está na raiz. O último elemento, 12, vai para o lugar dele.

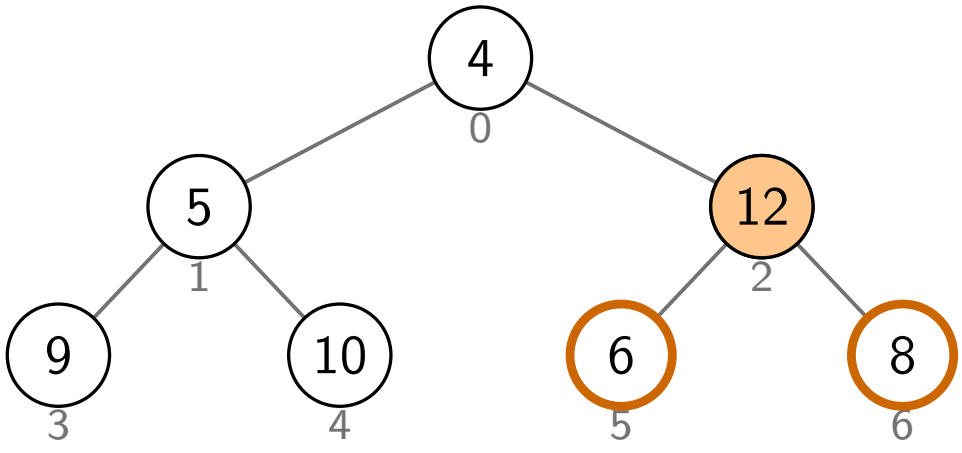
# Retirada do mínimo: o último elemento desce

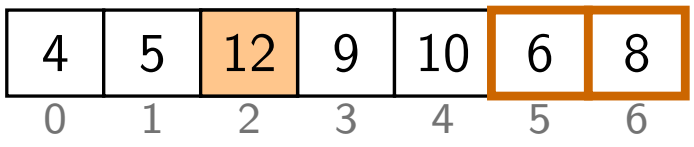



Os filhos do 12 são 5 e 4. O menor é o 4, e como $4 < 12$, os dois trocam.

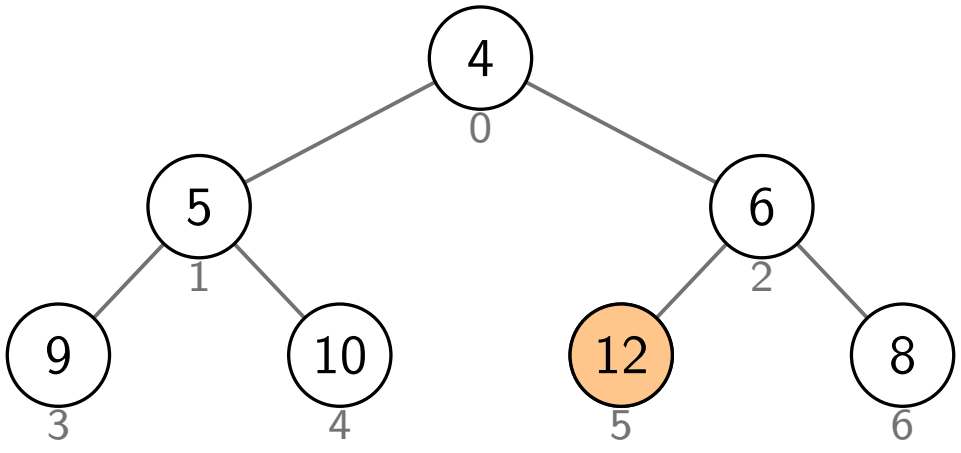
# Retirada do mínimo: o último elemento desce




Os filhos do 12 agora são 6 e 8. O menor é o 6, e como $6 < 12$, os dois trocam.

# Retirada do mínimo: o último elemento desce




| 4 | 5 | 6 | 9 | 10 | 12 | 8 |
|---|---|---|---|---|---|---|
| 0 | 1 | 2 | 3 | 4 | 5 | 6 |

O 12 chegou a uma folha e para.

O novo menor, 4, está na raiz.

# Retirada do mínimo em Go

```go
func (h *HeapMin) RetiraMin() int {
  menor := h.v[0]
  ultimo := len(h.v) - 1
  h.v[0] = h.v[ultimo]
  h.v = h.v[:ultimo]
  if len(h.v) > 0 {
    h.desce(0)
  }
  return menor
}
```

```go
func (h *HeapMin) desce(i int) {
  n := len(h.v)
  for {
    menor := i
    esq, dir := 2*i+1, 2*i+2
    if esq < n && h.v[esq] < h.v[menor] {
      menor = esq
    }
    if dir < n && h.v[dir] < h.v[menor] {
      menor = dir
    }
    if menor == i {
      return
    }
    h.v[menor], h.v[i] = h.v[i], h.v[menor]
    i = menor
  }
}
```

O último elemento vai para a raiz e troca de lugar com o menor dos filhos enquanto for maior que ele. Também são no máximo $\lfloor \log_2 n \rfloor$ trocas.

# Fila de prioridade com `container/heap`

O pacote faz o sobe e o desce. O tipo informa o tamanho, a comparação e a troca, e como acrescentar e retirar do fim do *slice*.

```
// O pacote container/heap organiza o slice como heap de mínimo.
type FilaArestas []Aresta

func (f FilaArestas) Len() int { return len(f) }
func (f FilaArestas) Less(i, j int) bool { return f[i].Peso < f[j].Peso }
func (f FilaArestas) Swap(i, j int) { f[i], f[j] = f[j], f[i] }
func (f *FilaArestas) Push(x any) { *f = append(*f, x.(Aresta)) }
func (f *FilaArestas) Pop() any {
  antigo := *f
  n := len(antigo)
  a := antigo[n-1]
  *f = antigo[:n-1]
  return a
}
```

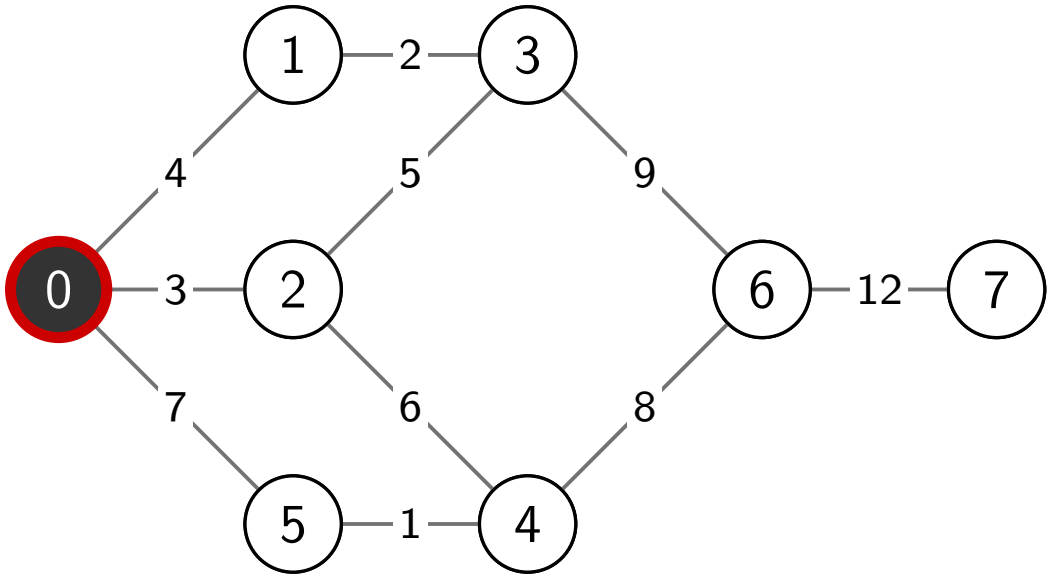
O programa usa `heap.Push(fila, a)` e `heap.Pop(fila)`, e não os métodos diretamente. No trabalho prático, `container/*` é proibido.

# Prim em Go

```
func Prim(g *GrafoP, s int) []Aresta {
  naArvore := make([]bool, g.V())
  fila := &FilaArestas{}
  var arvore []Aresta

  // v entra na árvore, e as
  // arestas de v para fora dela,
  // na fila.
  visita := func(v int) {
    naArvore[v] = true
    for _, a := range g.adj[v] {
      if !naArvore[a.Outro(v)] {
        heap.Push(fila, a)
      }
    }
  }
  visita(s)
  for fila.Len() > 0 &&
    len(arvore) < g.V()-1 {
    a := heap.Pop(fila).(Aresta)
    if naArvore[a.V] && naArvore[a.W] {
      continue // descarta
    }
    arvore = append(arvore, a)
    novo := a.V
    if naArvore[a.V] {
      novo = a.W
    }
    visita(novo)
  }
  return arvore
}
```

# Prim passo a passo



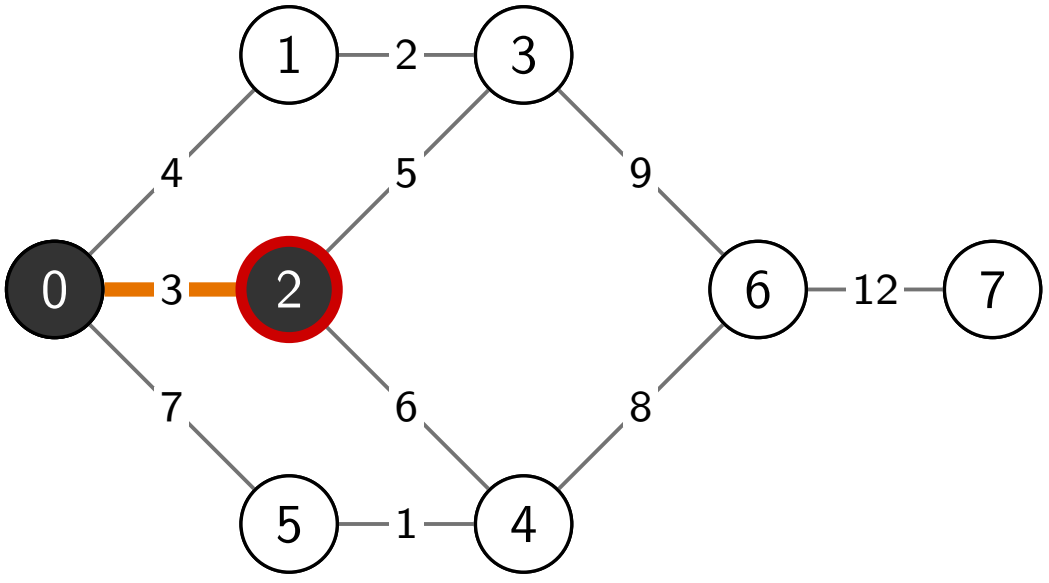

fila de prioridade

| 0-2 (3) |
|---|
| 0-1 (4) |
| 0-5 (7) |

início: 0 entra na árvore, e as arestas de 0 vão para a fila

na árvore — entra agora — descartada: os dois extremos na árvore

# Prim passo a passo



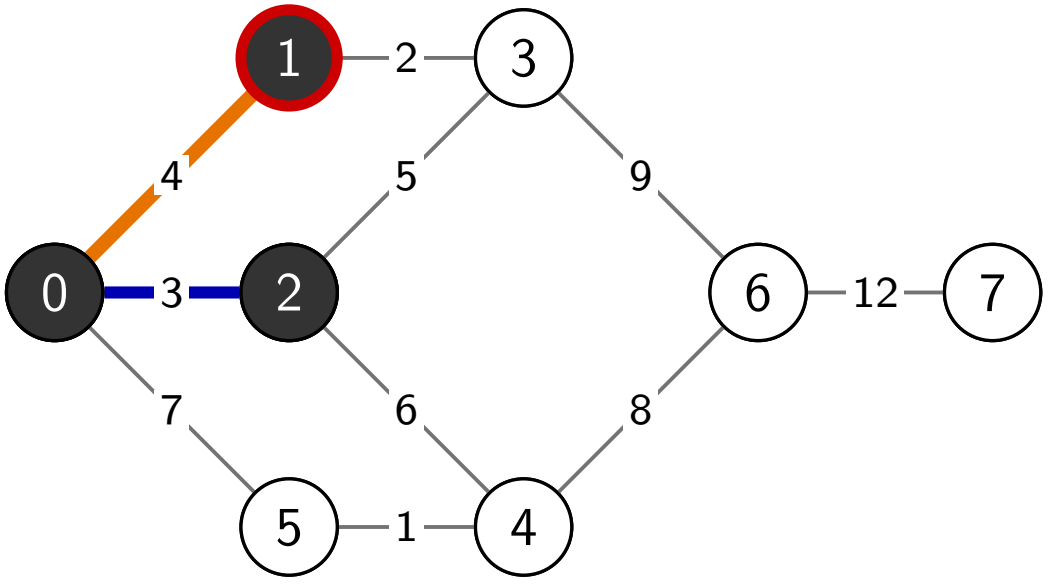

fila de prioridade

| 0-1 (4) |
|---|
| 2-3 (5) |
| 2-4 (6) |
| 0-5 (7) |

retira 0-2, peso 3: 2 entra, e 2-3 e 2-4 vão para a fila

na árvore — entra agora — descartada: os dois extremos na árvore

# Prim passo a passo



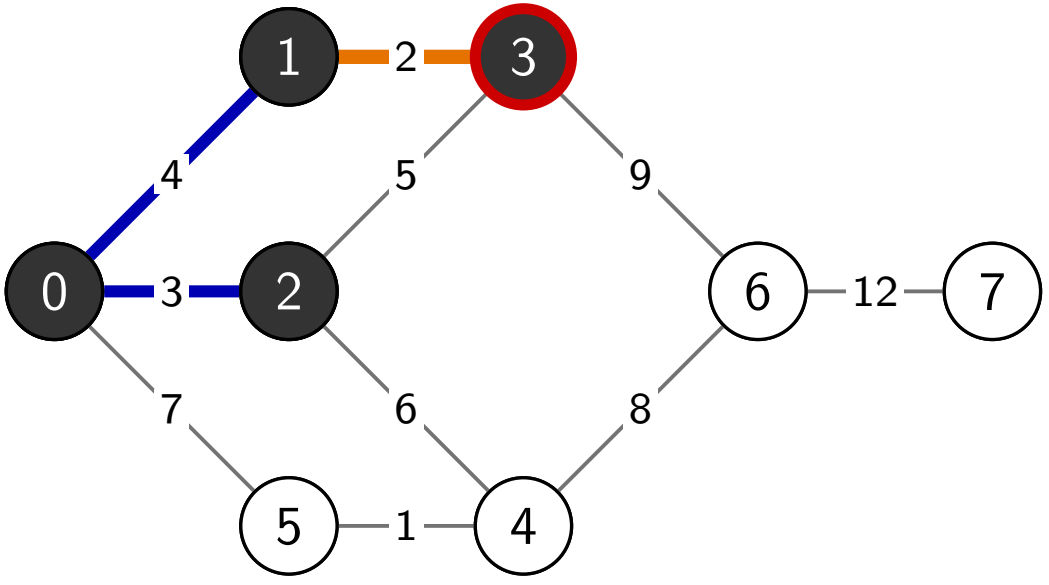

fila de prioridade

| 1–3 (2) |
|---|
| 2–3 (5) |
| 2–4 (6) |
| 0–5 (7) |

retira 0-1, peso 4: 1 entra, e 1-3 vai para a fila

na árvore  entra agora  descartada: os dois extremos na árvore

# Prim passo a passo



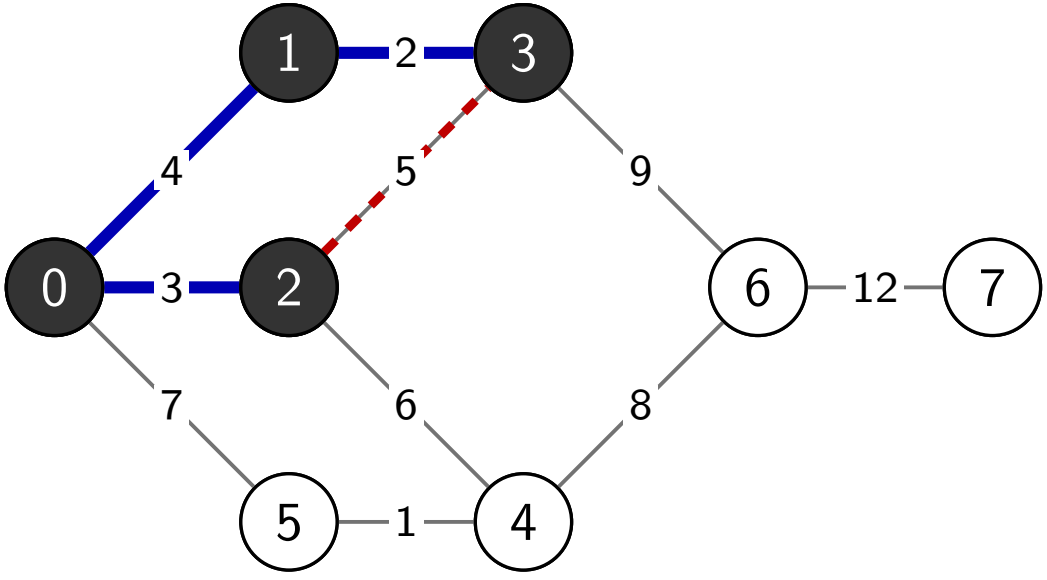

fila de prioridade

| |
|---|
| 2–3 (5) |
| 2–4 (6) |
| 0–5 (7) |
| 3–6 (9) |

retira 1-3, peso 2: 3 entra, e 3-6 vai para a fila

na árvore — entra agora — descartada: os dois extremos na árvore

# Prim passo a passo



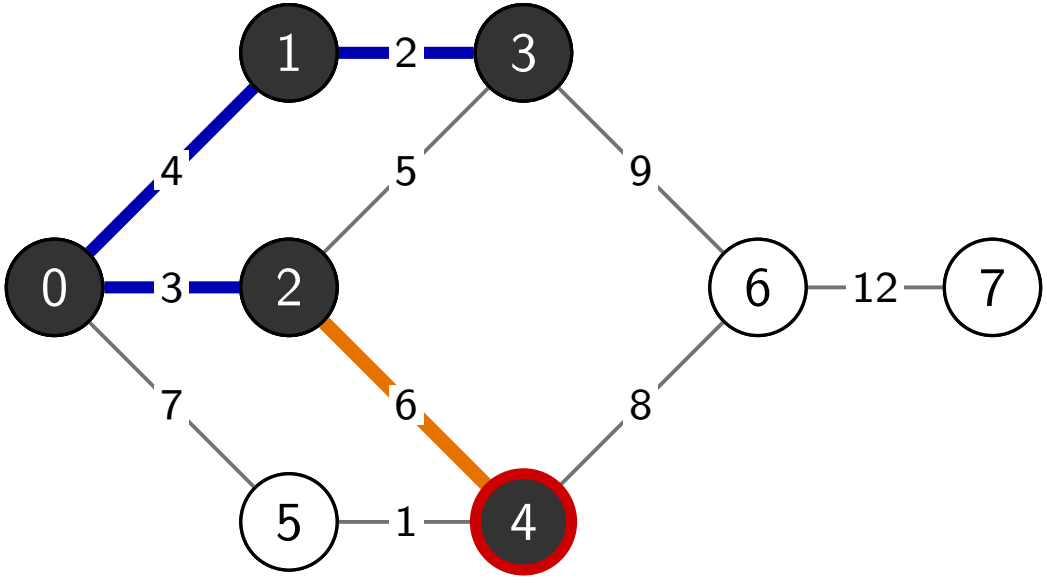

fila de prioridade

| 2–4 (6) |
|---|
| 0–5 (7) |
| 3–6 (9) |

retira 2-3, peso 5: 2 e 3 já estão na árvore, descarta

na árvore — entra agora — descartada: os dois extremos na árvore

# Prim passo a passo



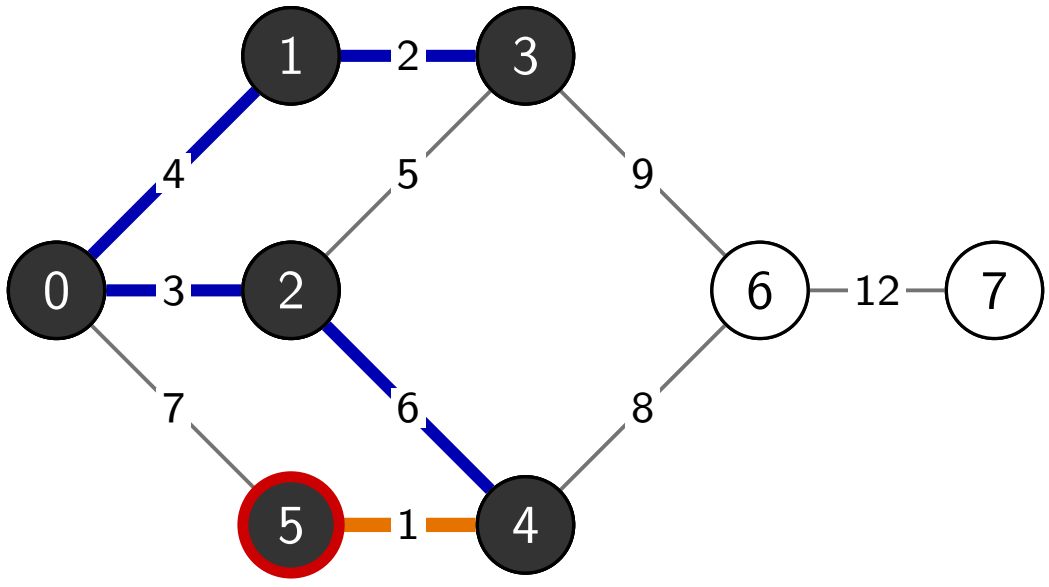

fila de prioridade

| 4–5 (1) |
|---|
| 0–5 (7) |
| 4–6 (8) |
| 3–6 (9) |

retira 2-4, peso 6: 4 entra, e 4-5 e 4-6 vão para a fila

na árvore  entra agora  descartada: os dois extremos na árvore

# Prim passo a passo



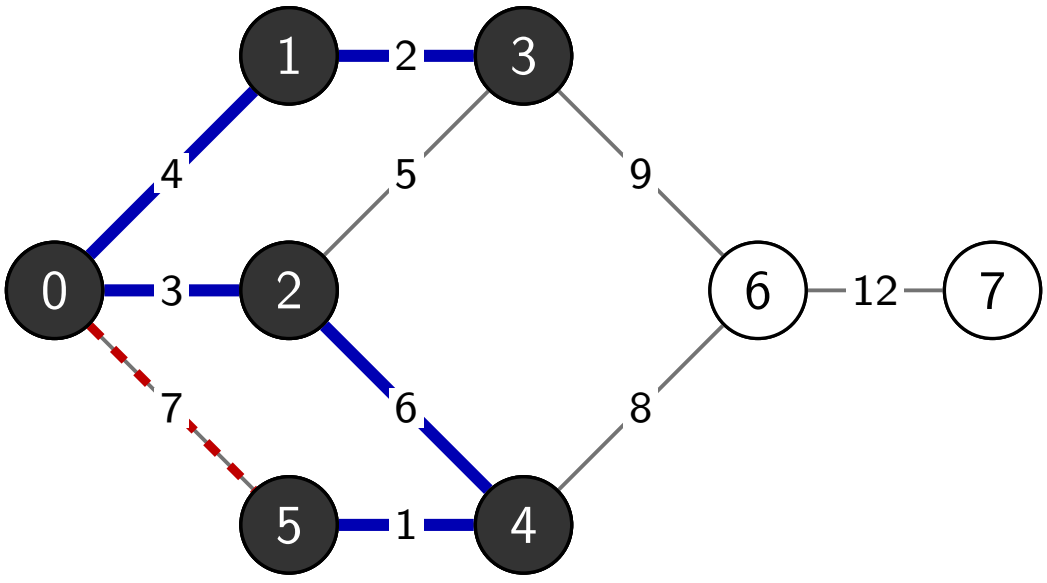

fila de prioridade

| 0–5 (7) |
|---|
| 4–6 (8) |
| 3–6 (9) |

retira 4-5, peso 1: 5 entra, sem arestas novas para a fila

na árvore — entra agora — descartada: os dois extremos na árvore

# Prim passo a passo



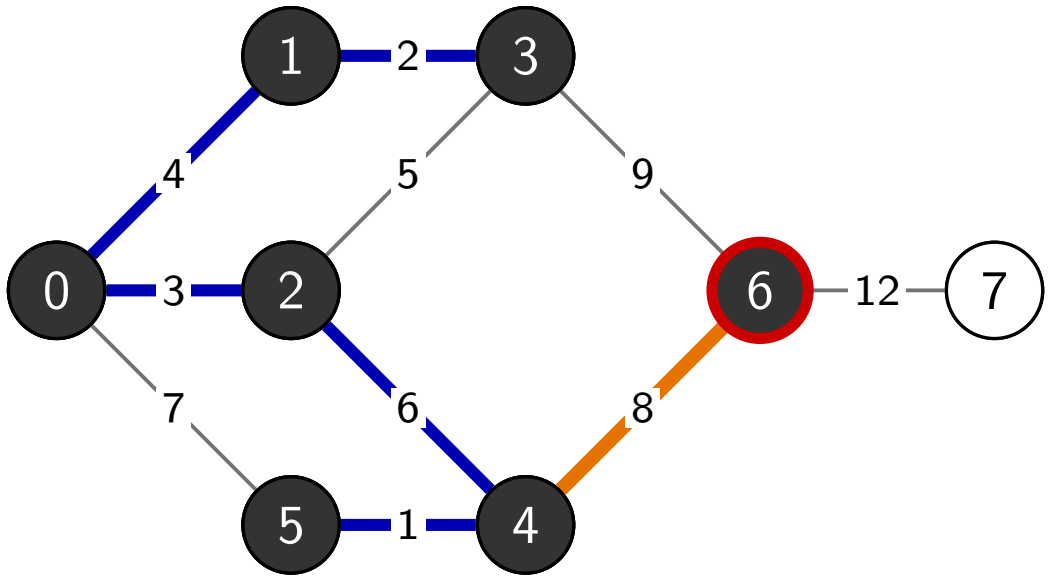

fila de prioridade

| 4–6 (8) |
|---|
| 3–6 (9) |

retira 0-5, peso 7: 0 e 5 já estão na árvore, descarta

na árvore — entra agora — descartada: os dois extremos na árvore

# Prim passo a passo



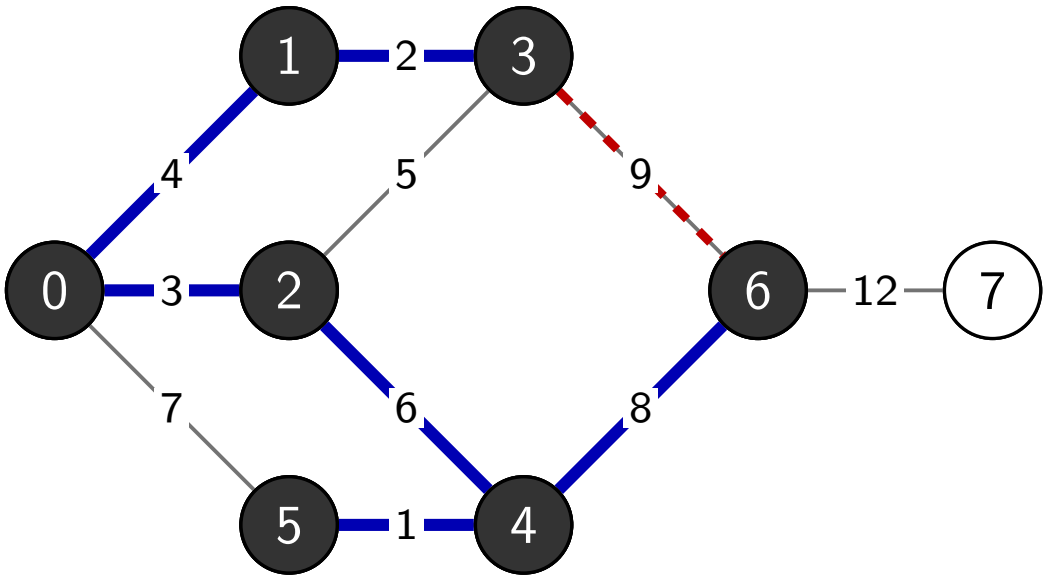

fila de prioridade

| 3–6 (9) |
|---|
| 6–7 (12) |

retira 4-6, peso 8: 6 entra, e 6-7 vai para a fila

na árvore  entra agora  descartada: os dois extremos na árvore

# Prim passo a passo



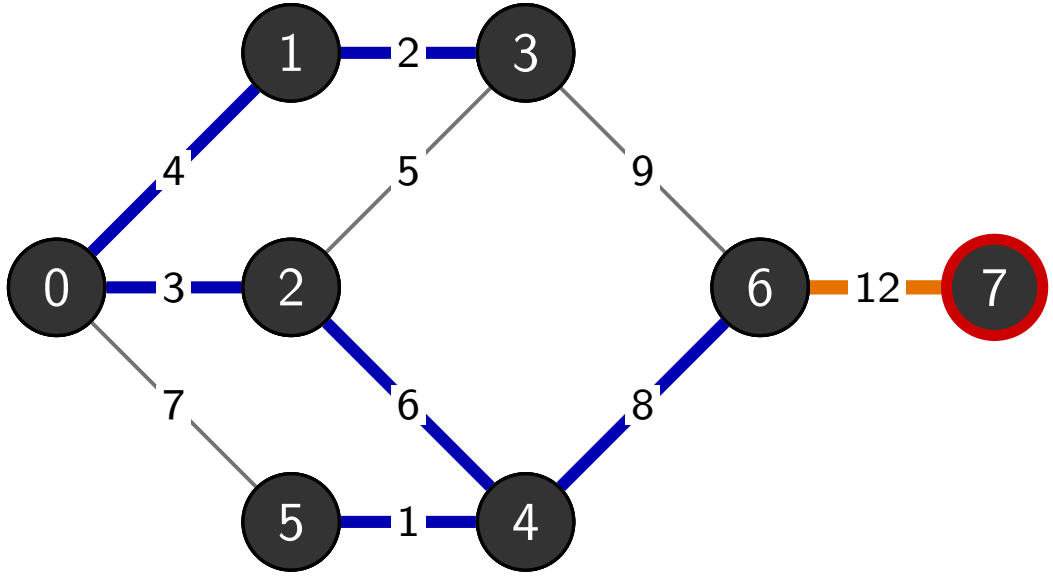

fila de prioridade

| 6–7 (12) |
|---|

retira 3-6, peso 9: 3 e 6 já estão na árvore, descarta

na árvore — entra agora — descartada: os dois extremos na árvore

# Prim passo a passo




fila de prioridade

retira 6-7, peso 12: 7 entra; árvore completa, fim

na árvore  entra agora  descartada: os dois extremos na árvore

# Prim em $G$, a partir de 0

```
$ go run prim.go
Prim a partir de 0
  início             fila: 0-2(3) 0-1(4) 0-5(7)
  0-2  3 entra 2     fila: 0-1(4) 2-3(5) 2-4(6) 0-5(7)
  0-1  4 entra 1     fila: 1-3(2) 2-3(5) 2-4(6) 0-5(7)
  1-3  2 entra 3     fila: 2-3(5) 2-4(6) 0-5(7) 3-6(9)
  2-3  5 descarta    fila: 2-4(6) 0-5(7) 3-6(9)
  2-4  6 entra 4     fila: 4-5(1) 0-5(7) 4-6(8) 3-6(9)
  4-5  1 entra 5     fila: 0-5(7) 4-6(8) 3-6(9)
  0-5  7 descarta    fila: 4-6(8) 3-6(9)
  4-6  8 entra 6     fila: 3-6(9) 6-7(12)
  3-6  9 descarta    fila: 6-7(12)
  6-7 12 entra 7     fila:

Árvore geradora mínima: 0-2 0-1 1-3 2-4 4-5 4-6 6-7
7 arestas, peso total 36
```

A mesma árvore do Kruskal, em outra ordem. A aresta 4-5, a mais leve, é a quinta a entrar.

# Custo do Prim

- cada aresta entra na fila no máximo uma vez e sai no máximo uma vez: $O(E \log E)$, com até $E$ arestas na fila;
- esta é a versão **preguiçosa**: arestas que deixaram de servir ficam na fila e são descartadas quando saem;
- a versão **ansiosa** guarda na fila um vértice por vez, com a melhor aresta conhecida até ele: $O(E \log V)$, com no máximo $V$ itens na fila;
- em grafo denso, um vetor no lugar da fila dá $O(V^2)$, sem heap.

# Kruskal ou Prim

| | Kruskal | Prim |
|---|---|---|
| o que cresce | uma floresta, que vira árvore no fim | uma árvore, a partir de $s$ |
| estrutura auxiliar | ordenação e Union-Find | fila de prioridade |
| custo | $O(E \log E)$ | $O(E \log E)$, ou $O(E \log V)$ na versão ansiosa |
| grafo desconexo | floresta geradora mínima | só a componente de $s$ |

Como $E < V^2$, $\log E < 2 \log V$: $O(E \log E)$ e $O(E \log V)$ diferem por uma constante.

# Origem do problema

- 1926, Otakar Borůvka: rede elétrica da Morávia, na atual República Tcheca, com o menor comprimento de linhas;
- 1930, Vojtěch Jarník: o algoritmo que hoje se chama de Prim;
- 1956, Joseph Kruskal; 1957, Robert Prim, nos Bell Labs, para redes telefônicas;
- 1959, Edsger Dijkstra redescobre o algoritmo de Jarník e Prim.

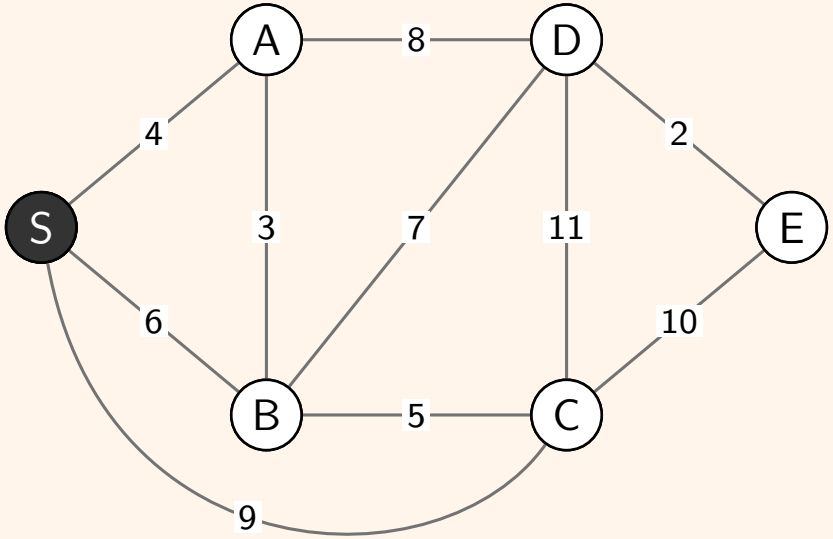
ODS 9, Indústria, Inovação e Infraestrutura:

9.1 infraestrutura de qualidade, confiável, sustentável e **resiliente**, com acesso equitativo e a preços acessíveis;

9.c aumentar o acesso às tecnologias de informação e comunicação e oferecer acesso universal à internet.

# Prática: a rede das escolas

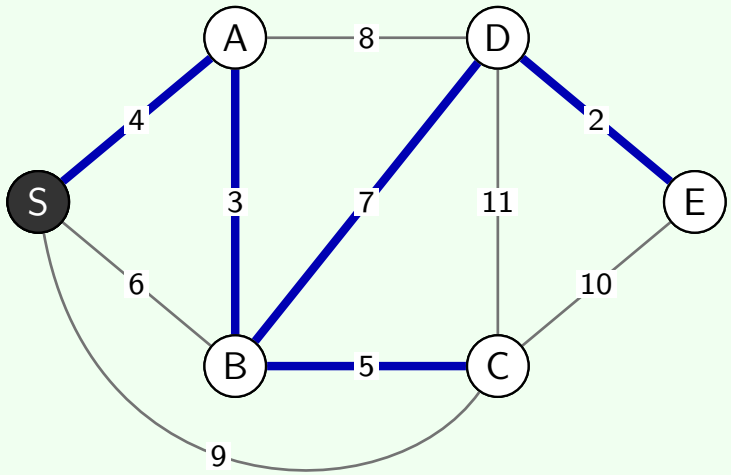
Custo de cada ligação possível, em km de cabo. A sede S já tem conexão.




1. execute o Kruskal à mão: liste as arestas na ordem em que são examinadas e marque as que entram e as que são descartadas;
2. calcule o custo total da rede.

Em dupla. Tempo: 7 minutos.

# Resposta: a rede das escolas




D-E 2 entra
A-B 3 entra
S-A 4 entra
B-C 5 entra
S-B 6 descarta
B-D 7 entra: 5 arestas, fim

Custo: $2 + 3 + 4 + 5 + 7 = 21$ km.

A-D, S-C, C-E e C-D nem chegam a ser examinadas.

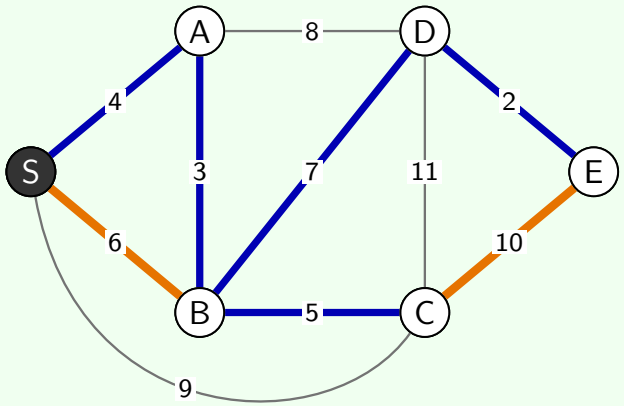
# Prática: e se um cabo for cortado?

Na rede de 21 km:

1. qual ligação, se for cortada, deixa mais escolas sem conexão com a sede? Quantas?
2. acrescente uma única ligação, a mais barata possível, de modo que nenhum corte isole mais de duas escolas;
3. quais ligações é preciso acrescentar para que nenhum corte isole escola alguma? Qual o custo total da rede?

Em dupla. Tempo: 8 minutos.

# Resposta: e se um cabo for cortado?




1. S-A: as cinco escolas;
2. S-B, 6 km. O pior corte passa a ser B-D, que isola D e E;
3. S-B e C-E: 16 km a mais, rede de 37 km, 76% mais cara.

Na árvore, toda aresta é uma ponte: retirada, desconecta o grafo. A redundância custa caro. A rede mais barata sem ponte custa 35 km, e não contém A-B nem B-D: partir da árvore e acrescentar arestas não dá o ótimo.

# O que o modelo deixa de fora

- **resiliência**: a árvore mínima não tem nenhuma redundância, e a meta 9.1 pede infraestrutura resiliente;
- **pontos de junção**: com três escolas nos vértices de um triângulo de 10 km de lado, a árvore mínima usa 20 km; uma caixa de emenda no centro liga as três com 17,3 km. Escolher os pontos extras é o problema da árvore de Steiner, NP-difícil;
- **custo real**: travessia de rio, licença ambiental, poste já existente. O peso da aresta é uma decisão de modelagem;
- **quem fica de fora**: a árvore liga todos os vértices que estão no grafo. Quem decide quais escolas entram é o projeto, e não o algoritmo.

# Onde aparecem as árvores geradoras mínimas

| Aplicação | O que a árvore representa |
|---|---|
| projeto de redes | elétrica, telefônica, de dados, de água, viária |
| agrupamento (*single-link*) | tirar as $k-1$ arestas mais pesadas da árvore deixa $k$ grupos |
| segmentação de imagens | pixels como vértices, diferença de cor como peso |
| caixeiro-viajante | a árvore dá uma rota no máximo duas vezes maior que a ótima, com pesos que obedecem à desigualdade triangular |
| taxonomia e biologia | árvores de semelhança entre espécies ou genes |

Lista baseada nos slides de Sedgewick e Wayne, *Minimum Spanning Trees*.

# De volta às escolas

```
Ligações da árvore e escolas isoladas da sede se a ligação for cortada:
  A-B   5.4 km   A
  B-K   5.7 km   F G K
  C-D   5.7 km   D
  C-E   7.2 km   C D
  E-J   7.3 km   J
  F-G   7.3 km   G
  F-K   8.2 km   F G
  S-H   8.2 km   H I
  H-I   8.5 km   I
  S-E   8.9 km   C D E J
  S-B   9.2 km   A B F G K
```

- 66 ligações possíveis, Kruskal sobre o grafo completo;
- o corte de S-B deixa cinco escolas sem conexão.

# Para a próxima aula

- exercícios do roteiro da aula, sem entrega e sem nota. O conteúdo cai na Prova 2;
- rodar `kruskal.go`, `prim.go` e `escolas.go`, trocando as arestas e os locais;
- leitura: Sedgewick, seção 4.3; Cormen, capítulo 23.

Próxima aula, 11/11: caminhos mínimos, com a Atividade Avaliativa 4. O algoritmo de Dijkstra tem a estrutura do Prim, com outra prioridade na fila. No mesmo dia vence a etapa 1 do trabalho prático.